

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_c5395511-084\agent-aa7f445d88b40de54.jsonl`
- SHA-256: `c46f4c0429d654e6a60c103addab0b6184ce54c8cb690ff8389780e02888ce46`
- Source modified: `2026-07-06T21:30:21+00:00`
- Imported at: `2026-07-08T16:00:10+00:00`
- project: `wf\_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:29:35.077Z)

Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there, or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (db._apply_schema on aiosqlite ":memory:").

WHAT F2 DOES:

- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video / _content_item_to_video / get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences (json.loads(policy.config_json)); TopicPreferenceStage (Tier 2): exclude-veto, include any/all, casefold substring match over source.title + content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty); _content_item_preview(...) builds a throwaway ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn, account, source, item) loads the default policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now: matched=engine.evaluate(); if matched and source: run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a /set (F1's merge) and are read via the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate: ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium (edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if handled / can't occur /

pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity.
FINDING (design_tests): F1 orchestrator evaluate_pipeline is dead in production; F2 hand-wires TopicPreferenceStage directly, so no unified StageResult trace is built
severity low | src/tg_video_filter/policy.py:176
summary: ADR 0012 F1 added evaluate_pipeline as the stage orchestrator, but F2 calls TopicPreferenceStage().evaluate() directly from _evaluate_topic_preferences and never routes through evaluate_pipeline, so the pipeline abstraction and its ordered StageResult explanation list stay unused outside of one unit test.
scenario: As later Track F phases (security_checks, deep_inspect) land, each will be tempted to hand-wire itself the same way; the FilterEngine result and the topic StageResult are merged ad hoc (on_new_message appends only topic_result.reason to a local reasons list) rather than accumulated by evaluate_pipeline, so there is no single ordered trace of why an item passed/failed, and the design diverges from the ADR 0012 pipeline model before it is ever used.
evidence: grep evaluate_pipeline over src/ finds only the definition (policy.py:176) and its __all__ entry (196) ? no call site. telethon_client.py:433 constructs a PolicyContext and calls TopicPreferenceStage().evaluate() directly. FilterEngine is not a PolicyStage, so the two evaluators cannot share evaluate_pipeline without wrapping the filter engine as a stage.
rec: Either route topic evaluation through evaluate_pipeline (wrapping FilterEngine as a PolicyStage) so the StageResult trace is real, or disclose in the PR/ADR that evaluate_pipeline is deferred and stages are hand-wired for F2. At minimum note the divergence so F3+ does not copy the hand-wiring pattern.

2. assistant (2026-07-06T21:29:38.297Z)

I'll adversarially verify this finding. Let me read the cited code and check the claims.

3. user (2026-07-06T21:29:40.379Z)

```
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:176:async def evaluate_pipeline(  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:196:     "evaluate_pipeline",  
Binary file C:/Users/avidu/Projects/_wt-  
pr51/src/tg_video_filter/__pycache__/policy.cpython-313.pyc matches
```

4. user (2026-07-06T21:29:41.646Z)

```
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:56:class PolicyStage(Protocol):  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:117:class TopicPreferenceStage:  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:176:async def evaluate_pipeline(  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:178:     stages:  
list[PolicyStage],  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:192:     "PolicyStage",  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:194:     "TopicPreferenceStage",  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/policy.py:196:     "evaluate_pipeline",  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/telethon_client.py:47:from  
tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage  
C:/Users/avidu/Projects/_wt-pr51/src/tg_video_filter/telethon_client.py:433:     return await  
TopicPreferenceStage().evaluate(  
Binary file C:/Users/avidu/Projects/_wt-  
pr51/src/tg_video_filter/__pycache__/policy.cpython-313.pyc matches  
Binary file C:/Users/avidu/Projects/_wt-  
pr51/src/tg_video_filter/__pycache__/telethon_client.cpython-313.pyc matches  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:10:     TopicPreferenceStage,  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:11:     evaluate_pipeline,  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:71:async def  
test_evaluate_pipeline_passes_all_stages_in_order() -> None:  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:78:     passed, results = await  
evaluate_pipeline(_ctx(), stages)  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:85:async def  
test_evaluate_pipeline_short_circuits_on_first_failure() -> None:  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:93:     passed, results = await  
evaluate_pipeline(_ctx(), stages)  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:101:async def  
test_evaluate_pipeline_with_no_stages_passes_vacuously() -> None:  
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:102:     passed, results = await
```

```

evaluate_pipeline(_ctx(), [])
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:118:# --- TopicPreferenceStage
-----
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:135:      result = await
TopicPreferenceStage().evaluate(_ctx(text="anything"))
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:147:      result = await
TopicPreferenceStage().evaluate(ctx)
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:160:      result = await
TopicPreferenceStage().evaluate(ctx)
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:173:      result = await
TopicPreferenceStage().evaluate(ctx)
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:188:      result = await
TopicPreferenceStage().evaluate(ctx)
C:/Users/avidu/Projects/_wt-pr51/tests/test_policy.py:196:      result = await
TopicPreferenceStage().evaluate(
Binary file C:/Users/avidu/Projects/_wt-
pr51/tests/__pycache__/test_policy.cpython-313-pytest-8.4.2.pyc matches

```

5. assistant (2026-07-06T21:29:44.479Z)

The finding claims `evaluate\_pipeline` has no call site and F2 hand-wires
`TopicPreferenceStage().evaluate()`. Let me read the relevant code sections.

6. user (2026-07-06T21:29:45.998Z)

```

50         reason=reason,
51         stage_name=stage_name,
52         metadata=metadata,
53     )
54
55
56     class PolicyStage(Protocol):
57         """A single stateless gate in the policy pipeline."""
58
59         async def evaluate(self, ctx: PolicyContext) -> StageResult:
60             """Evaluate ``ctx`` and return a stage result."""
61             ...
62
63
64     @dataclass(frozen=True)
65     class TopicPreferences:
66         """Parsed ADR 0012 topic preference config."""
67
68         include: tuple[str, ...] = ()
69         exclude: tuple[str, ...] = ()
70         mode: str = "any"
71
72         @property
73         def enabled(self) -> bool:
74             return bool(self.include or self.exclude)
75
76
77     def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
78         """Parse ``topics`` from a raw policy config object."""
79         raw_topics = config.get("topics")
80         if raw_topics is None:
81             return TopicPreferences()
82         if not isinstance(raw_topics, dict):
83             raise ValueError("topics must be a JSON object")
84         mode = raw_topics.get("mode", "any")
85         if mode not in ("any", "all"):
86             raise ValueError("topics.mode must be 'any' or 'all'")
87         return TopicPreferences(
88             include=_string_tuple(raw_topics.get("include")),
89             exclude=_string_tuple(raw_topics.get("exclude")),

```

```

90         mode=str(mode),
91     )
92
93
94 def load_topic_preferences(policy: Policy) -> TopicPreferences:
95     """Load topic preferences from ``policy.config_json``."""
96     decoded = json.loads(policy.config_json)
97     if not isinstance(decoded, dict):
98         raise ValueError("policy config_json must decode to a JSON object")
99     return parse_topic_preferences(decoded)
100
101
102 def _string_tuple(value: object) -> tuple[str, ...]:
103     if value is None:
104         return ()
105     if not isinstance(value, list):
106         raise ValueError("topic include/exclude values must be arrays")
107     topics: list[str] = []
108     for item in value:
109         if not isinstance(item, str):
110             raise ValueError("topic include/exclude arrays must contain strings")
111         topic = item.strip()
112         if topic:
113             topics.append(topic)
114     return tuple(topics)
115
116
117 class TopicPreferenceStage:
118     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
119
120     stage_name = "topic-preferences"
121
122     async def evaluate(self, ctx: PolicyContext) -> StageResult:
123         try:
124             prefs = load_topic_preferences(ctx.policy)
125         except (json.JSONDecodeError, ValueError) as exc:
126             return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
127         if not prefs.enabled:
128             return StageResult.pass_(self.stage_name, "topic preferences disabled")
129
130         haystack = _topic_haystack(ctx)
131         matched_excludes = _matched_topics(prefs.exclude, haystack)
132         if matched_excludes:
133             return StageResult.fail(
134                 self.stage_name,
135                 f"excluded topic matched: {matched_excludes[0]}",
136                 matched_exclude_topics=list(matched_excludes),
137             )
138
139         if not prefs.include:
140             return StageResult.pass_(self.stage_name, "no include topics configured")
141
142         matched_includes = _matched_topics(prefs.include, haystack)
143         if prefs.mode == "any":
144             if matched_includes:
145                 return StageResult.pass_(
146                     self.stage_name,
147                     f"matched include topic: {matched_includes[0]}",
148                     matched_include_topics=list(matched_includes),
149                 )
150             return StageResult.fail(self.stage_name, "no include topics matched")
151
152         missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
153         if missing:

```

```

154         return StageResult.fail(
155             self.stage_name,
156             f"missing required topic: {missing[0]}",
157             matched_include_topics=list(matched_includes),
158             missing_include_topics=list(missing),
159         )
160     return StageResult.pass(
161         self.stage_name,
162         "matched all include topics",
163         matched_include_topics=list(matched_includes),
164     )
165
166
167 def _topic_haystack(ctx: PolicyContext) -> str:
168     parts = [ctx.source.title or "", ctx.content_item.text or ""]
169     return "\n".join(parts).casefold()
170
171
172 def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173     return tuple(topic for topic in topics if topic.casefold() in haystack)
174
175
176 async def evaluate_pipeline(
177     ctx: PolicyContext,
178     stages: list[PolicyStage],
179 ) -> tuple[bool, list[StageResult]]:
180     """Evaluate stages in order and short-circuit on the first failure."""
181     results: list[StageResult] = []
182     for stage in stages:
183         result = await stage.evaluate(ctx)
184         results.append(result)
185         if not result.passed:
186             return False, results
187     return True, results
188
189
190 __all__ = [
191     "PolicyContext",
192     "PolicyStage",
193     "StageResult",
194     "TopicPreferenceStage",
195     "TopicPreferences",
196     "evaluate_pipeline",
197     "load_topic_preferences",
198     "parse_topic_preferences",
199 ]

```

7. user (2026-07-06T21:29:46.709Z)

```

410         provider=source.provider,
411         document_id=str(document_id) if document_id is not None else None,
412         source_id=source.id,
413         provider_message_id=str(message_id),
414         size_bytes=meta.size_bytes,
415         duration_s=meta.duration_s,
416         height=meta.height,
417         width=meta.width,
418     ),
419     matched=matched,
420 )
421
422
423 async def _evaluate_topic_preferences(
424     conn: aiosqlite.Connection,
425     account_id: int,

```

```

426         source: Source,
427         content_item: ContentItem,
428     ) -> StageResult | None:
429         """Evaluate ADR 0012 topic preferences for a candidate item."""
430         policy = await db.get_default_policy(conn, account_id)
431         if policy is None:
432             return None
433         return await TopicPreferenceStage().evaluate(
434             PolicyContext(content_item=content_item, source=source, policy=policy)
435         )
436
437
438     def _peer_id(peer: object) -> int:
439         """Best-effort numeric id for the peer of a message.
440
441         Telegram represents channel/group/user peers as different TL types; we
442         collapse them to a single int for dedup/storage. The exact subclass isn't
443         important ? only that the id is stable per chat.
444         """
445         # Each Peer* type has a field like channel_id / chat_id / user_id.
446         for field in ("channel_id", "chat_id", "user_id"):
447             val = getattr(peer, field, None)
448             if val is not None:
449                 return int(val)

```

8. assistant (2026-07-06T21:29:52.087Z)

The finding's factual claims are all confirmed by the code:

- `evaluate\_pipeline` (policy.py:176) is defined and exported, but the only references in `src/` are the definition and the `\_\_all\_\_` entry. No call site.
- `telethon\_client.py:433` calls `TopicPreferenceStage().evaluate()` directly, constructing a `PolicyContext` inline, not routing through `evaluate\_pipeline`.
- `evaluate\_pipeline` is only exercised in `tests/test\_policy.py` (3 tests).

Now let me assess the severity claim. The finding self-rates as **low** (hygiene/scope-note). Let me verify whether this is actually a defect in THIS PR's diff or a pre-existing/out-of-scope situation. `evaluate\_pipeline` was added by F1 (a prior PR), not this PR. Let me check the diff.

9. user (2026-07-06T21:29:54.477Z)

```

diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index e7d0303..395435a 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -7,6 +7,7 @@ the first failure. Later Track F phases add concrete stages.

```

```

    from __future__ import annotations

+import json
    from dataclasses import dataclass, field
    from typing import Protocol

@@ -60,6 +61,118 @@ class PolicyStage(Protocol):
    ...

+@dataclass(frozen=True)
+class TopicPreferences:
+    """Parsed ADR 0012 topic preference config."""
+
+    include: tuple[str, ...] = ()
+    exclude: tuple[str, ...] = ()
+    mode: str = "any"
+

```

```

+     @property
+     def enabled(self) -> bool:
+         return bool(self.include or self.exclude)
+
+
+def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
+    """Parse ``topics`` from a raw policy config object."""
+    raw_topics = config.get("topics")
+    if raw_topics is None:
+        return TopicPreferences()
+    if not isinstance(raw_topics, dict):
+        raise ValueError("topics must be a JSON object")
+    mode = raw_topics.get("mode", "any")
+    if mode not in ("any", "all"):
+        raise ValueError("topics.mode must be 'any' or 'all'")
+    return TopicPreferences(
+        include=_string_tuple(raw_topics.get("include")),
+        exclude=_string_tuple(raw_topics.get("exclude")),
+        mode=str(mode),
+    )
+
+
+def load_topic_preferences(policy: Policy) -> TopicPreferences:
+    """Load topic preferences from ``policy.config_json``."""
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_topic_preferences(decoded)
+
+
+def _string_tuple(value: object) -> tuple[str, ...]:
+    if value is None:
+        return ()
+    if not isinstance(value, list):
+        raise ValueError("topic include/exclude values must be arrays")
+    topics: list[str] = []
+    for item in value:
+        if not isinstance(item, str):
+            raise ValueError("topic include/exclude arrays must contain strings")
+        topic = item.strip()
+        if topic:
+            topics.append(topic)
+    return tuple(topics)
+
+
+class TopicPreferenceStage:
+    """ADR 0012 Tier 2 topic matching over source title and captured caption."""
+
+    stage_name = "topic-preferences"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            prefs = load_topic_preferences(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
+        if not prefs.enabled:
+            return StageResult.pass_(self.stage_name, "topic preferences disabled")
+
+        haystack = _topic_haystack(ctx)
+        matched_excludes = _matched_topics(prefs.exclude, haystack)
+        if matched_excludes:
+            return StageResult.fail(
+                self.stage_name,
+                f"excluded topic matched: {matched_excludes[0]}",
+                matched_exclude_topics=list(matched_excludes),
+            )

```

```

+         )
+
+     if not prefs.include:
+         return StageResult.pass_(self.stage_name, "no include topics configured")
+
+     matched_includes = _matched_topics(prefs.include, haystack)
+     if prefs.mode == "any":
+         if matched_includes:
+             return StageResult.pass_(
+                 self.stage_name,
+                 f"matched include topic: {matched_includes[0]}",
+                 matched_include_topics=list(matched_includes),
+             )
+         return StageResult.fail(self.stage_name, "no include topics matched")
+
+     missing = tuple(topic for topic in prefs.include if topic not in matched_includes)
+     if missing:
+         return StageResult.fail(
+             self.stage_name,
+             f"missing required topic: {missing[0]}",
+             matched_include_topics=list(matched_includes),
+             missing_include_topics=list(missing),
+         )
+     return StageResult.pass_(
+         self.stage_name,
+         "matched all include topics",
+         matched_include_topics=list(matched_includes),
+     )
+
+def _topic_haystack(ctx: PolicyContext) -> str:

```

10. assistant (2026-07-06T21:30:00.105Z)

Confirmed: `evaluate\_pipeline` is NOT part of this PR's diff ? it was added by F1. This PR adds `TopicPreferenceStage` but does not touch `evaluate\_pipeline`. So the "dead in production" observation about `evaluate\_pipeline` is a pre-existing condition (F1 landed it unused).

The genuinely PR-specific part of the finding: F2 introduces the *first* concrete stage and *chooses* to hand-wire it via `TopicPreferenceStage().evaluate()` rather than through `evaluate\_pipeline`. That is a real decision made in this diff (telethon_client.py:433). Let me verify how the result is consumed in `on\_new\_message` to confirm the "ad hoc merge / no unified trace" characterization.

11. user (2026-07-06T21:30:02.011Z)

```

269-
270-     text = _message_text(message)
271-     result = engine_holder[ENGINE_KEY].evaluate(meta)
272-     matched = result.matched
273-     reasons = list(result.reasons)
274-     if matched and source is not None:
275-         topic_result = await _evaluate_topic_preferences(
276-             conn,
277-             user.id,
278-             source,
279-             _content_item_preview(
280-                 text,
281-                 matched,
282-             ),
283-         )
284-     if topic_result is not None:
285-         reasons.append(topic_result.reason)
286-         matched = topic_result.passed
287-

```

```

292-
293-         record = VideoRecord(
294-             user_id=user.id,
295-             channel_id=channel_id,
296-             --
319-             document_id,
320-         )
321-         return
322-     log.info(
323-         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
324-         user.id,
325-         channel_id,
326-         message_id,
327-         matched,
328-         reasons,
329-     )
330-
331-     if matched:
332-         # Build link text for LINK/BOTH modes (None for FORWARD).
333-         --
419-         matched=matched,
420-     )
421-
422-
423:async def _evaluate_topic_preferences(
424-    conn: aiosqlite.Connection,
425-    account_id: int,
426-    source: Source,
427-    content_item: ContentItem,
428-    --
467-    "build_client",
468-    "extract_video_meta",
469-    "make_new_message_handler",
470-    "_content_item_preview",
471-    "_evaluate_topic_preferences",
472-    "_message_text",
473-]

```

12. assistant (2026-07-06T21:30:12.660Z)

The characterization is accurate: ``on_new_message`` (telethon_client.py:270-291) runs the FilterEngine, then hand-wires the topic stage, appends only ``topic_result.reason`` to a local ``reasons`` list, and overwrites ``matched``. There is no ``evaluate_pipeline``-produced ordered ``StageResult`` trace; the two evaluators are merged ad hoc. The FilterEngine is not a ``PolicyStage``, so it cannot currently share ``evaluate_pipeline``.

Assessment:

****What's true:**** Every factual claim holds. ``evaluate_pipeline`` has zero production call sites; F2's stage is hand-wired at telethon_client.py:433; results are merged ad hoc in `on_new_message`; FilterEngine isn't a `PolicyStage`.

****What weakens it as a "defect in THIS PR's diff":****

1. ``evaluate_pipeline`` being unused is a ****pre-existing**** F1 condition, not introduced by this PR (confirmed via the diff ? this PR does not touch it).
2. There is ****no wrong runtime behavior****. Gating is functionally correct (main reviewer verified veto/include/all). This is purely an architectural/design-consistency observation.
3. The finding itself is framed as a scope-note ("disclose the divergence" / "at minimum note it so F3+ doesn't copy it") ? a hygiene recommendation, not a bug.

This lands as a design-consistency note, not a defect that ships wrong behavior. The finding is essentially correct in substance but is a low-severity architectural observation whose central artifact (``evaluate_pipeline`` dead) predates this PR. The PR-specific decision (hand-wiring the first concrete stage) is real and worth noting for F3+, so it's not fully refuted ? it's a genuine, correctly-rated low-severity scope note.

13. user (2026-07-06T21:30:21.410Z)

Structured output provided successfully